# A CASE-STUDY BASED REPORT

**ON**

# Phishing Website Detection Using Random Forest and NLP-Based URL Analysis

*submitted in partial fulfilment of the requirements for the Course*

## MACHINE LEARNING

## BACHELOR OF TECHNOLOGY

in

**A.Y.: 2026-27**

**by**

**Venkat Siddharth Reddy Moku (Roll No. 2520090026)**

**Sai Saketh Vutharavalli (Roll No. 2520090055)**

*Under the Esteemed guidance of*

**Dr. Y. Lakshmi Prasanna**

Asst. Professor, Dept. of CS&IT




Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

# INDEX

Project documentation based on the repository snapshot inspected on 14 October 2026.


| Section |
|---|
| Abstract |
| 1. Introduction |
| 1.1 Background and motivation |
| 1.2 Project overview |
| 1.3 Tools and technology stack |
| 1.4 Scope and responsible use |
| 2. Problem statement and objectives |
| 3. Dataset and exploratory analysis |
| 3.1 PhiUSIIL dataset profile |
| 3.2 Data cleaning and leakage controls |
| 3.3 URL feature engineering |
| 4. Algorithm and system design |
| 4.1 Random Forest classifier |
| 4.2 End-to-end architecture |
| 4.3 Backend API and frontend |
| 5. Implementation, evaluation and tests |
| 5.1 Recorded V3 model performance |
| 5.2 Dataset bias and V4 experiments |
| 5.3 Automated API test coverage |
| 5.4 Reproducibility and execution |
| 6. Conclusion and future work |
| References |

# Project Team

Project Title: Phishing Website Detection Using Random Forest and NLP-Based URL Analysis

| Team member | Roll number |
|---|---|
| Venkat Siddharth Reddy Moku | 2520090026 |
| Sai Saketh Vutharavalli | 2520090055 |

Project Guide: Dr. Y. Lakshmi Prasanna, Assistant Professor, Department of CS&IT

| Name / role | Signature | Date |
|---|---|---|
| Venkat Siddharth Reddy Moku | _____ | _____ |
| Sai Saketh Vutharavalli | _____ | _____ |
| Project Guide | _____ | _____ |

Note on implementation scope: The administrative project title is retained from the title page. The inspected classifier is Random Forest over engineered URL lexical/statistical features; the implementation does not use TF-IDF, bag-of-words, word embeddings, or an NLP text-classification pipeline. An optional Groq explanation service exists in the backend but does not determine or override the classifier label.

# Abstract

Phishing URLs are crafted to resemble trusted online services and can lead users to credential theft, payment fraud, or malware delivery. This project, implemented as PhishGuard, applies a supervised Random Forest classifier to features derived from the structure of a submitted URL. The repository contains the PhiUSIIL Phishing URL Dataset, training and evaluation scripts, serialized model artifacts, a FastAPI backend, a React/TypeScript frontend, and automated API tests.

The active backend code loads the Random Forest V3 model and constructs a 21-value feature vector from the URL without visiting the destination website. Features include URL and domain length, IP-address usage, subdomain count, percent-encoding/obfuscation indicators, counts and ratios of letters, digits and special characters, HTTPS status, character continuation rate, and lookup-based statistics. The code excludes raw high-cardinality identifiers and the dataset field URLSimilarityIndex, which is documented as a target-leakage risk.

The V3 metadata records 97.61% accuracy on a held-out set of 47,284 samples, with 99.06% phishing precision, 95.28% phishing recall and a 97.13% phishing F1-score. These are repository-reported metrics; they were not independently rerun as part of preparing this document. The repository also includes separate V4 experiments for modern-hosting representation bias, but its backend still loads V3. The system is a decision-support tool, not a guarantee of safety, and URL-only inference can produce false positives or miss novel attacks.

Keywords: Phishing URL detection, Random Forest, URL feature engineering, cybersecurity, PhiUSIIL, FastAPI, React, evaluation metrics.

# 1. Introduction

## 1.1 Background and motivation

Phishing is a social-engineering technique in which an attacker uses a deceptive message, URL, or website to induce a target to disclose credentials or perform an unsafe action. Because malicious URLs change frequently, a detector based only on a list of known bad domains can become outdated. Supervised machine learning can learn combinations of URL characteristics from labelled examples and provide a repeatable first-pass assessment.

The repository's development notes also identify a practical dataset problem: legitimate URLs with paths, queries, or newer hosted subdomains may look structurally unusual if those patterns are underrepresented in the training set. PhishGuard therefore includes feature engineering, API integration and separate experiments aimed at improving structural generalisation.

## 1.2 Project overview

PhishGuard processes a user-provided HTTP or HTTPS URL, validates its basic structure, derives the 21 production features, passes them to a trained Random Forest V3 classifier, and returns the label, predicted-class probability, phishing probability/risk score, and optionally a separate natural-language explanation. The frontend presents the result with a threat label, risk gauge, feature and model information, and service status.

### 1.3 Tools and technology stack

| Layer | Repository technologies / files | Purpose |
|---|---|---|
| Machine learning | Python; pandas; NumPy; scikit-learn; joblib | Dataset inspection, feature extraction, Random Forest training/evaluation, persistence. |
| Backend API | FastAPI; Uvicorn; Pydantic; python-dotenv | Input validation, model loading, health check, prediction and risk-score response. |
| Frontend | React 19; TypeScript; Vite; Tailwind CSS; Lucide React | URL submission and result dashboard. |
| Dataset | PhiUSIIL_Phishing_URL_Dataset.csv | Labelled source corpus for development and model experiments. |
| Tests | backend/test_api.py; FastAPI TestClient | API response, feature schema and input validation checks. |
| Optional explanation | groq_service.py; GROQ_API_KEY | Explanatory text only; does not change Random Forest output. |

### 1.4 Scope and responsible use

- The classifier uses engineered lexical/statistical URL features rather than NLP text vectorisation. No TF-IDF, bag-of-words, transformer, or word-embedding pipeline is present in the active classifier.
- The API does not fetch or render the target website. This reduces crawling risk and allows analysis of links that may be offline, but it also means page content, DNS reputation, redirects and live hosting signals are not examined.
- Treat “legitimate” as a model prediction, not an assurance that the destination is safe. High or uncertain risk should lead to caution and human review.
- Use datasets in accordance with their licences. Keep API keys in environment variables and do not commit .env files or credentials.
- Do not use the classifier as the sole control for security-sensitive decisions. Monitor errors and retrain only after controlled evaluation.

## 2. Problem Statement and Objectives

### 2.1 Case-study scenario

A student, organisation or support desk receives web links through email, chat or forms and needs a quick initial assessment before opening them. PhishGuard accepts a URL string and estimates whether its learned characteristics resemble legitimate or phishing examples. The service avoids automatically browsing the target site, limiting exposure to active malicious content.

### 2.2 Central question

Can a Random Forest model trained on engineered URL lexical and statistical attributes provide useful phishing classification while keeping inference fast, reproducible and independent of live webpage crawling?

### 2.3 Objectives

- Inspect and prepare the PhiUSIIL dataset and document its class distribution and duplicate URL records.
- Exclude raw string identifiers and the documented leakage-prone URLSimilarityIndex field from the core feature set.
- Extract 21 consistent URL-derived features and preserve their exact order between training and inference.
- Train and evaluate a Random Forest classifier with reproducible hyperparameters and held-out metrics.
- Expose the model through FastAPI endpoints and connect them to a React/TypeScript interface.
- Check normal and suspicious URL examples, malformed inputs, API readiness, and optional explanation fallback behaviour.
- Document dataset bias, false-positive/false-negative trade-offs and limits of URL-only classification.

### 2.4 Assumptions and limitations

The target labels in the dataset are treated as reference labels, although real-world labelling may be imperfect. Feature definitions and ordering must remain consistent with the saved model. The recorded evaluation depends on the reported split and dataset construction. Model probabilities represent learned class scores and do not prove actual website intent, reputation, or safety.

## 3. Dataset and Exploratory Analysis

### 3.1 PhiUSIIL dataset profile

The repository includes phishguard/dataset/PhiUSIIL_Phishing_URL_Dataset.csv. The project README reports that the raw file contains 235,795 rows and 55 columns, with 4 object/string columns and 51 numeric columns, and no missing cells. It reports 425 duplicate URL occurrences and 0 identical full-row duplicates. After dropping duplicate URLs, the V3 audit describes 235,370 unique records: 134,850 legitimate and 100,520 phishing.

| Dataset item | Repository-reported value | Interpretation |
|---|---|---|
| Raw rows / columns | 235,795 / 55 | Initial PhiUSIIL corpus dimensions. |
| Missing values | 0 across columns | No null cells reported during the dataset audit. |
| Duplicate URL occurrences | 425 | Duplicate URL records were removed before the documented clean dataset was formed. |
| Clean unique URL records | 235,370 | The V3 structural audit records this |

| Dataset item | Repository-reported value | Interpretation |
|---|---|---|
| | | total. |
| Legitimate class (label=1) | 134,850 (57.3% of clean records) | Class mapped to legitimate. |
| Phishing class (label=0) | 100,520 (42.7% of clean records) | Class mapped to phishing. |

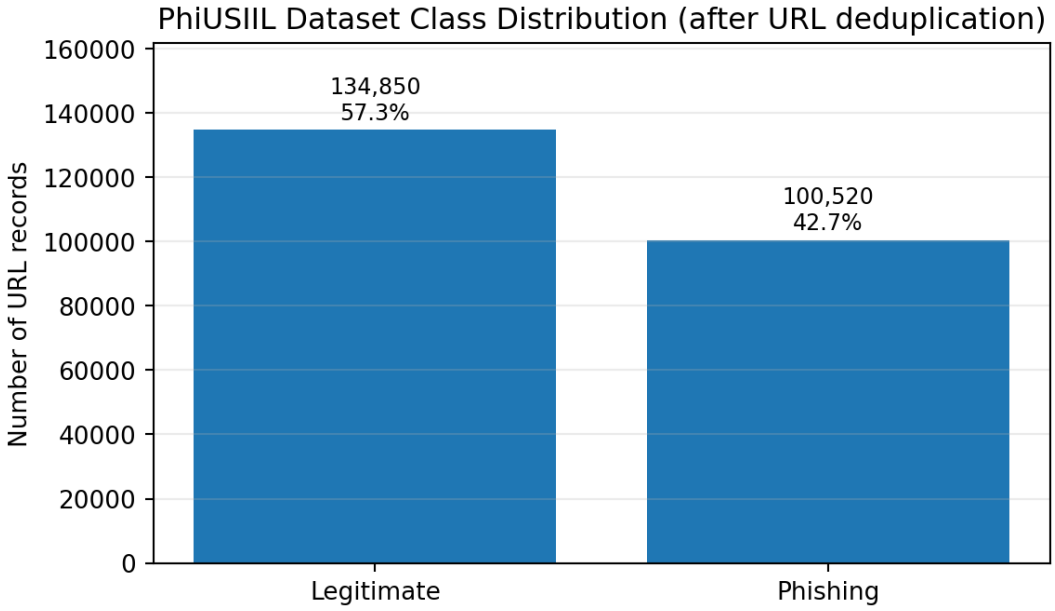



*Figure 1. Class balance reported by the V3 structural dataset audit (after URL deduplication).*

## 3.2 Data cleaning and leakage controls

The original feature_config.py explicitly separates raw identifiers, leakage-prone fields, URL lexical features and HTML-content features. The following controls are documented in the repository training pipeline:

- Deduplication: rows with duplicate URL values are reduced to one URL record before the described train/test split.
- Raw identifier exclusion: URL, Domain, TLD, and Title are not passed as raw text features because their high cardinality risks memorisation and they are not part of the no-NLP production model.
- Target leakage exclusion: URLSimilarityIndex is excluded. The repository reports it equals 100 for every legitimate row in its investigation, creating a shortcut that could inflate apparent performance.
- Domain-aware split in V3: training_metadata_v3.json records 0 overlap between train and held-out base domains, intended to reduce leakage from URLs sharing a domain.
- Class mapping: label 0 denotes phishing; label 1 denotes legitimate.

## 3.3 Exploratory findings and dataset bias

The structural audit reports a difference between class distributions in the source dataset. It records that 27.14% of phishing examples contain a path, compared with 0% of legitimate examples; 6.10% of phishing examples contain a question mark, compared with 0% of legitimate examples. It also reports

HTTPS for all legitimate examples in this corpus and about 49.12% of phishing examples. Those relationships may help separate these particular labelled samples, but they can also teach shortcuts that do not generalise to all current URLs.

| Feature (dataset audit) | Legitimate examples | Phishing examples | Caution |
|---|---|---|---|
| Mean URL length | 26.23 characters | 45.70 characters | Distributions overlap; some phishing URLs are short and some legitimate URLs are long. |
| Mean other special characters | 1.24 | 3.80 | Punctuation is a useful signal, not definitive proof. |
| HTTPS share | 100.00% | 49.12% | HTTPS does not mean a site is trustworthy; this is a dataset-specific pattern. |
| Has path | 0.00% | 27.14% | The source distribution may penalise ordinary legitimate deep links unless training examples cover them. |

Interpretation: these numbers describe the source dataset rather than the whole Internet. Dataset composition and evaluation design should therefore be documented alongside every model score.

# 4. Algorithm and System Design

## 4.1 Random Forest classifier

Random Forest is an ensemble of decision trees. Each tree is built from sampled training data and considers subsets of candidate features; the trees' votes are combined for the final class. This is suitable for tabular features such as URL length, digit counts, subdomain count and protocol indicators. The saved V3 metadata and backend source identify the classifier as scikit-learn RandomForestClassifier.

| Parameter | Recorded value | Purpose |
|---|---|---|
| n_estimators | 100 | Number of decision trees. |
| max_depth | None | No explicit maximum tree depth. |
| min_samples_split | 2 | Minimum samples needed to split a node. |
| min_samples_leaf | 1 | Minimum samples permitted at a leaf. |
| class_weight | balanced | Adjusts class contribution based on class frequency. |
| random_state | 42 | Supports repeatable training runs. |
| n_jobs | -1 | Uses available CPU cores for parallel work where supported. |

## 4.2 Production URL feature engineering

The active V3 extractor in phishguard/ml/v3/feature_config_v3.py standardises a leading www. prefix and trims trailing slashes for selected length/structural calculations. It does not fetch the destination or parse page HTML. The model input is a fixed ordered vector of 21 values:

| Feature | What it represents |
|---|---|
| URLLength | Length of the normalised URL string. |
| DomainLength | Length of canonical host string. |
| IsDomainIP | Whether host matches an IPv4 address pattern. |
| TLDLength | Length of final top-level domain component. |
| NoOfSubDomain | Count of subdomain levels under the extractor's rule. |
| HasObfuscation | Indicator for percent-encoded characters. |
| NoOfObfuscatedChar | Count of percent-encoded byte sequences. |
| ObfuscationRatio | Encoded characters relative to URL length. |
| NoOfLettersInURL | Alphabetic-character count after selected stripping. |
| LetterRatioInURL | Ratio of letters to URL length. |
| NoOfDegitsInURL | Digit count (feature name preserved from code). |
| DegitRatioInURL | Ratio of digits to URL length (feature name preserved from code). |
| NoOfEqualsInURL | Count of = characters. |
| NoOfQMarkInURL | Count of ? characters. |
| NoOfAmpersandInURL | Count of & characters. |
| NoOfOtherSpecialCharsInURL | Count of other punctuation/non-alphanumeric symbols. |
| SpacialCharRatioInURL | Ratio of selected special characters (feature name preserved from code). |
| IsHTTPS | Indicator for an https:// prefix. |
| CharContinuationRate | Normalised length of longest same character-type run. |
| TLDLegitimateProb | Lookup statistic associated with final domain suffix. |
| URLCharProb | Lookup statistic derived from character-frequency data. |

The spelling of fields such as NoOfDegitsInURL, DegitRatioInURL and SpacialCharRatioInURL follows the actual feature schema. The order is important: the API checks that the extracted vector agrees with the model's expected column names where those names are available.

### 4.3 End-to-end architecture

| User pastes URL | FastAPI validation | V3 feature extractor | Random Forest V3 | React result panel |
|---|---|---|---|---|
| Input | HTTP/HTTPS, host | 21 numeric features | Class + scores | Label + risk |

*Figure 2. Logical flow of the production application; destination sites are not crawled.*

Process sequence: (1) the frontend submits a URL; (2) Pydantic validation accepts only supported HTTP/HTTPS syntax with a hostname and applies a maximum length; (3) the V3 extractor builds the ordered vector; (4) the loaded Random Forest returns a class and class probabilities; (5) the API computes risk_score = round(P(phishing) × 100) and returns the response to the frontend. If configured, Groq generates a separate text explanation from supplied model outputs and extracted features, but its documented role is supplementary and it must not override the Random Forest result.

### 4.4 Backend endpoints

| Endpoint | Input / output | Role |
|---|---|---|
| GET /api/health | Returns status, model name, and model_loaded. | Checks API/model readiness. |
| POST /api/analyze | JSON request: {"url":"https://example.com"}; returns prediction, label, probability, risk_score, model_version, feature_count, phishing_probability and optional explanation fields. | Validates URL and runs inference. |
| GET /docs | FastAPI-generated Swagger / OpenAPI UI. | Development and API inspection. |

## 5. Implementation, Evaluation and Tests

### 5.1 Repository components inspected

| Component | Repository path | Verified purpose |
|---|---|---|
| Project README | phishguard/README.md | Overview, dataset audit, model notes and run instructions. |
| Feature configuration | phishguard/ml/feature_config.py; phishguard/ml/v3/feature_config_v3.py | Defines feature groups, label mapping and URL-only feature extractor. |
| Training/evaluation scripts | phishguard/ml/train.py; phishguard/ml/evaluate.py | V1 training and evaluation workflow. |
| Active model artefact | phishguard/ml/v3/random_forest_v3.pkl | The model path loaded by the current backend. |
| V3 metadata/report | phishguard/ml/v3/ training_metadata_v3.json; evaluation_report_v3.txt | Recorded V3 sample counts, test metrics, comparison and structural benchmark. |

| Component | Repository path | Verified purpose |
|---|---|---|
| API implementation | phishguard/backend/main.py | Loads V3, validates URLs, extracts features and returns prediction. |
| API tests | phishguard/backend/test_api.py | Fourteen unittest methods defined for schema, examples, invalid input and fallback. |
| Frontend | phishguard/frontend/src/ | URL scanner, result card, risk gauge, model/feature sections and API client. |
| V4 experiments | phishguard/ml/v4/ | Separate research artifacts on modern-hosting representation bias; not loaded by current API. |

## 5.2 Recorded V3 evaluation

phishguard/ml/v3/training_metadata_v3.json records a domain-grouped experiment with 233,086 training samples and 47,284 held-out test samples. The V3 comparison report notes 45,000 structurally augmented legitimate samples, so the V3 experiment total is not the same as the 235,370-row deduplicated base corpus used to describe the original dataset. Metadata records 0 overlapping base domains between training and holdout. The metrics below are copied from that metadata; this documentation process did not independently retrain the model or execute the tests.

| Metric | Recorded V3 result | Meaning |
|---|---|---|
| Accuracy | 97.61% | Share of held-out records classified correctly. |
| Phishing precision | 99.06% | Among records predicted as phishing, share labelled phishing. |
| Phishing recall | 95.28% | Among phishing records in holdout, share detected by model. |
| Phishing F1-score | 97.13% | Harmonic mean of phishing precision and recall. |
| False-positive rate on legitimate | 0.67% | Legitimate holdout URLs flagged as phishing. |
| False-negative rate on phishing | 4.72% | Phishing holdout URLs predicted legitimate. |

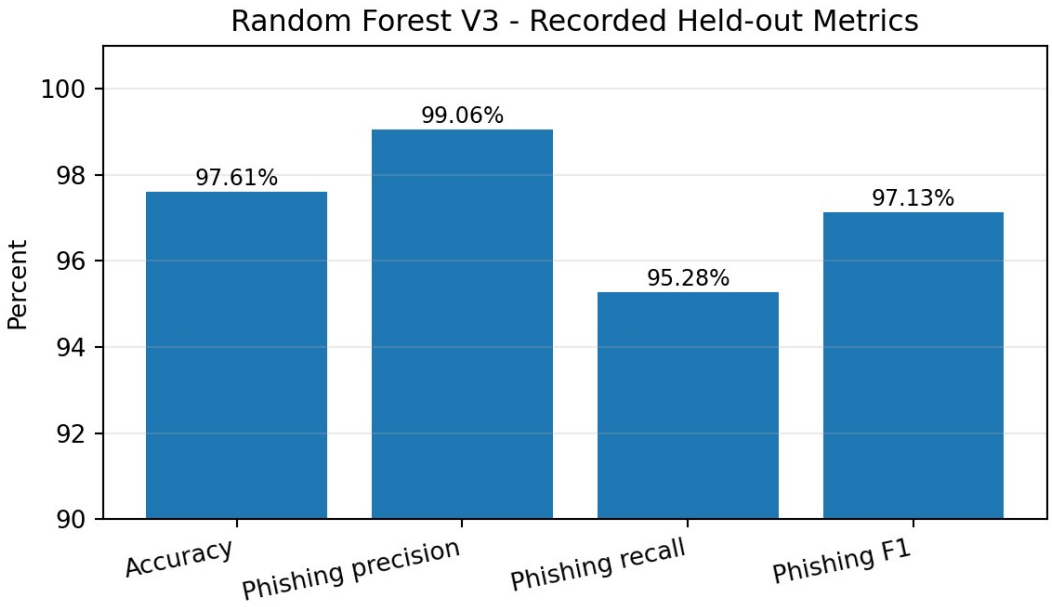



*Figure 3. V3 metrics recorded in the repository metadata (rounded to two decimals).*

## 5.3 Confusion matrix

The V3 metadata gives the following held-out confusion counts. Class 0 denotes phishing and class 1 denotes legitimate.

| Actual class | Predicted phishing | Predicted legitimate | Row total |
|---|---|---|---|
| Phishing (0) | 19,169 | 949 | 20,118 |
| Legitimate (1) | 182 | 26,984 | 27,166 |
| Total | 19,351 | 27,933 | 47,284 |

The 949 phishing false negatives are the more direct security concern, because they represent phishing-labelled holdout examples not caught by this test. The 182 legitimate false positives represent friction for legitimate users. Neither figure should be read as a guarantee of equal performance on future URL populations.

## 5.4 Earlier baseline and V4 experiments

The repository contains multiple model generations. The original Phase 1 report records 99.6644% accuracy for a V1 21-feature URL-only model under its original stratified split; that score is not the model path used by the current backend. The V3 metadata instead reports 97.61% on its domain-aware held-out experiment. These figures are not directly interchangeable because the experiment construction differs.

| Model / artefact | Accuracy reported | Status in repository |
|---|---|---|
| V1 URL-only baseline | 99.66% | Earlier baseline in ml/evaluation_report.txt; not loaded by current backend. |
| V3 production path | 97.61% | Active model path in backend/main.py; metric from V3 metadata. |
| V4-A experiment | 97.93% | Experimental metrics only; not loaded by current backend. |
| V4-B experiment | 97.56% | Experimental metrics only; not loaded by current backend. |

V4 notes describe an attempt to address representation bias in modern hosting platforms by adding verified legitimate examples alongside platform-based phishing samples and testing domain-grouped splits. The V4 report explicitly states that V3 remains the active production model. V4 metrics are experimental and should not be described as deployed performance unless the API model path is changed and revalidated.

## 5.5 Targeted structural benchmark

The V3 comparison report documents a separate 46-case structural benchmark covering legitimate apex URLs, www variants, paths, queries, fragments, multi-subdomains, phishing lures, obfuscated URLs, IP-based URLs and long phishing URLs. The report states V3 passed all 46 cases. Because this is a small, purpose-built test set, 100% on that benchmark does not imply 100% real-world accuracy; the larger held-out evaluation is a different measurement.

## 5.6 Automated tests and security checks

The repository contains backend/test_api.py with 14 unittest methods. The source covers: exact 21-feature names and ordering; the health endpoint; representative legitimate URLs (ChatGPT, Google, GitHub, Wikipedia and Python documentation); phishing-lure URLs; raw-IP URLs; malformed or unsupported schemes; response schema for the optional AI explanation; and graceful fallback when GROQ_API_KEY is absent. These tests are present in code, but execution was not performed as part of writing this report, so no passing status is asserted here.

## 5.7 Limitations

- URL-only features cannot observe page content, certificates, DNS, redirects, brand impersonation context or newly compromised legitimate websites.

- Feature extraction includes lookup-based suffix and character statistics from the project corpus; unseen or rare patterns may be represented imperfectly.
- IsHTTPS and punctuation-related features may encode dataset shortcuts. The dataset audit indicates major differences between legitimate and phishing URL structures.
- Probability/risk scores are model outputs, not calibrated guarantees. A low score must not be used as proof that a site is benign.
- Optional Groq text explanations require a configured external API key and network access. They are supplementary; the Random Forest remains the authoritative classifier in the code comments.
- V1, V3 and V4 results come from different saved experimental pipelines and must always be labelled with their model version and split methodology.

## 5.8 Reproducibility and execution

The following commands are taken from the repository READMEs. Run them from a suitable Python and Node.js environment. The repository README recommends Node.js 18+ (it also notes testing with Node 24).

### Train and evaluate the Phase 1 baseline

```
cd phishguard/ml
python train.py
python evaluate.py
```

### Start the FastAPI backend

```
cd phishguard/backend
pip install -r requirements.txt
python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

### Start the frontend in a separate terminal

```
cd phishguard/frontend
npm install
npm run dev
```

By default the frontend README documents a Vite development address at http://127.0.0.1:5173/ and the backend at http://127.0.0.1:8000. Confirm environment variable VITE_API_BASE_URL points at the intended backend. To regenerate the exact active V3 metrics, follow the V3-specific training/evaluation instructions or scripts in phishguard/ml/v3/; do not assume that running the Phase 1 root-level scripts reproduces V3.

# 6. Conclusion and Future Work

PhishGuard is implemented as a full-stack phishing URL assessment prototype. The inspected code shows a Random Forest V3 model using 21 engineered URL lexical/statistical features, a FastAPI service that performs feature extraction and inference without visiting destination URLs, a React/TypeScript interface, and a defined backend test suite. The V3 metadata records 97.61% held-out accuracy, 99.06% phishing precision, and 95.28% phishing recall across 47,284 held-out records with zero reported base-domain overlap.

The main technical lesson from the repository is that high apparent baseline performance can coexist with dataset representation problems. V3 uses a more cautious domain-aware evaluation and documents false positives in modern hosting contexts; V4 experiments explore data augmentation and

feature-prior changes, but those experimental artefacts are not the model loaded by the current API. For an academic demonstration, report model version, dataset construction and test methodology together with the metric values.

### Future work

- Rerun the V3 training/evaluation pipeline in the target environment and archive console output, configuration and package versions.
- Run and capture the output of the 14 API unit tests before final submission.
- Expand independent and time-separated testing, including legitimate modern hosted projects and confirmed phishing examples, with careful label verification.
- Review calibration and thresholds; measure precision-recall trade-offs on a realistic deployment distribution.
- Consider deferring or removing optional Groq explanations where a strictly local, Random-Forest-only deployment is required.
- Keep V4 experimental results separated from the active model until a controlled release is selected, evaluated and deployed.

## References

1. Project repository and source code snapshot (main branch, commit b53930b4c0763240b4fa83799d613bb9c5d59522, accessed 14 October 2026). https://github.com/gingermafoe/-Phishing-Website-Detection-Using-Random-Forest-and-NLPBased-URL-Analysis.
2. Scikit-learn: RandomForestClassifier documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
3. Scikit-learn: Model evaluation and classification metrics. https://scikit-learn.org/stable/modules/model_evaluation.html
4. FastAPI documentation. https://fastapi.tiangolo.com/
5. React documentation. https://react.dev/
6. PhiUSIIL Phishing URL Dataset, included in the project repository as phishguard/dataset/PhiUSIIL_Phishing_URL_Dataset.csv. Dataset attribution and licensing should be verified from the original dataset source before redistribution.

### Documentation note

This report is based on the repository files and saved metadata visible at the inspected commit. Code paths and README statements are described as repository contents, not as proof that the system was executed during this documentation task. Before submission, run the project locally, verify model loading and API responses, execute the tests, and update figures or records if the output differs.